

Governed AI Software Construction: Typed Graph Functions, Evidence-Bearing Execution, and Recursive Product Development

Dimitar Popov

Version 0.1 - research draft, 13 July 2026

Abstract

Large language models can now modify repositories, run tools, diagnose failures, and complete increasingly complex software-engineering tasks. Most systems place this capability inside an agent harness: the model receives a task, selects actions, edits files, observes tool output, and iterates until a test or stopping condition is reached. This has produced substantial gains on issue-resolution benchmarks, but it leaves a different engineering question open: what makes an AI-authored change part of a governed software product rather than a plausible patch accepted by an orchestration loop?

This paper presents ABIOgenesis, a reference architecture for governed probabilistic software construction. The architecture separates four concerns that are commonly collapsed. Constitutional specification states what product claims are authoritative and how change may lawfully re-enter them. GTL provides an LLM-first, graph-first typed algebra in which constructive workflows are published as graph functions rather than hidden in prompts or controllers. ABG interprets admitted GTL programs through an event-sourced runtime whose state, lineage, evidence, continuation, correction, and closure are replay-derived. Product and domain layers own meaning, policy, and acceptance interpretation without acquiring a second execution engine or event authority.

The core runtime unit is a typed traversal from an input contract to an output contract under deterministic, probabilistic, and human evaluation regimes. Probabilistic work may propose artifacts and semantic judgments, but those outputs become runtime facts only through admission. Deterministic checks, probabilistic evaluation, and human authority remain distinct; none silently impersonates another. Every admitted transition preserves intent lineage and accounts for carried obligations as realized, refined, deferred, blocked, repriced, or retained as open pressure. Runtime history is corrected by new events rather than erased, and current state is a projection over that history.

The paper also reports a negative architectural result from development of the 5.0 line. An attempted self-hosting fixed-point design would have proved only that two runtimes could package already-authored frozen source equivalently. It could not prove that the system authored its successor. The target was repriced: ABIOgenesis 5.0 is the stable, source-independent baseline; recursive dogfooding begins only when installed 5.0 and an independently released GLC product are used

as the development product for the 5.0.1 source project. This separates release qualification from successor self-use and turns the latter into a future empirical exposure rather than retroactive evidence.

Version 0.1 defines the system model, relates it to software-engineering agents and harness research, records the course correction, and specifies an evaluation programme. It does not yet claim benchmark superiority, a completed 5.0 release, or successful recursive dogfooding.

Contents

1	Reading Contract	4
1.1	The paper is a system description with open empirical exposure	4
1.2	The internal names are not the contribution by themselves	5
2	The Problem: Patch Success Is Not Product Accountability	5
2.1	Software agents optimize the act of changing code	5
2.2	Agent orchestration often becomes accidental product law	6
2.3	The research question	6
3	Contribution	7
3.1	A language/runtime split for probabilistic construction	7
3.2	An event-sourced epistemic runtime	7
3.3	Regime separation and admission authority	8
3.4	Obligation-conserving traversal and lawful re-entry	8
3.5	Recursive product identity without bootstrap conflation	8
3.6	The architecture at one cut	9
4	Related Work	9
4.1	Repository-level issue resolution	9
4.2	Role-based and multi-agent software construction	9
4.3	Harness engineering and intent-centric software engineering	10
4.4	Requirements, traceability, and provenance	10
4.5	Conceptual lineage	10
4.6	The claimed gap	11
5	Constitutional Specification	11
5.1	Specification is active product authority	11
5.2	Change begins at the smallest lawful re-entry point	12
5.3	Specification constrains the AI and the human	12
6	GTL: The Constructive Algebra	12
6.1	Graph is the structural type	12
6.2	Graph functions carry constructive meaning	13
6.3	The compute algebra is distinct from graph topology	13
6.4	Programs are configuration, not engine branches	14

6.5	Robustness and composability define an atom	14
7	ABG: The Evidence-Bearing Runtime	14
7.1	Events are the only admitted write path	14
7.2	The traversal unit	15
7.3	The admitted compute cycle	15
7.4	Deterministic, probabilistic, and human fibres	15
7.5	Proposal, evaluation, verification, and admission are separate roles	16
7.6	Closure is conjunctive	16
7.7	Correction preserves history	16
8	Obligation Conservation and Re-entry	16
8.1	Intent lineage is part of compute	16
8.2	Obligation topology may be discovered during construction	17
8.3	Gaps are typed outcomes	17
9	Product, Install, and Development Product	17
9.1	Four identities must not collapse	17
9.2	Source independence is a release property	17
9.3	Catalog products extend meaning without extending authority	18
10	The Self-Hosting Course Correction	18
10.1	The original ambition	18
10.2	The packaging fixed point	18
10.3	Campaign authoring and its different proof	18
10.4	The stable-first resolution	19
10.5	The substrate/subject rule	19
11	Current Product Realization	19
11.1	Implemented predecessor surface	19
11.2	Immutable predecessor evidence	19
11.3	Active 5.0 realization	20
11.4	Excluded product claims	20
12	Evaluation Programme	21
12.1	Evaluation principle	21
12.2	Core experiments	21
12.3	Metrics	22
12.4	Promotion law	22
13	Threats to Validity and Limitations	23
13.1	Single-project development	23
13.2	Test volume is not claim coverage	23
13.3	Vocabulary and structural overhead	23
13.4	Event truth is not world truth	23
13.5	Trusted-desktop boundary	23

13.6	Incomplete current result	23
13.7	No claim about model cognition	23
14	Discussion	24
14.1	The harness becomes a language implementation	24
14.2	Governance is runtime structure	24
14.3	Reliability belongs to the composed system	24
14.4	Recursive dogfooding is an architectural test	24
15	Conclusion	24
16	Appendix A. Claim Ledger	25
17	Appendix B. Current Authority and Evidence Map	26
18	References	26

1 Reading Contract

1.1 The paper is a system description with open empirical exposure

This manuscript combines a realized predecessor system, a current product contract, and a preregistered successor experiment. Those are different kinds of evidence and are marked accordingly.

- **Implemented** means the named behavior exists on the TypeScript product line and has repository-local executable evidence. Final publication claims still require an immutable release artifact and its qualification bundle.
- **Specified** means the behavior is authoritative product truth but may still be under design, realization, or release qualification.
- **Historical result** means an earlier design or experiment occurred and is retained as evidence, including negative evidence. It is not necessarily current product behavior.
- **Design hypothesis** means the architecture predicts a useful property that has not yet been established by a controlled comparison.
- **Open exposure** means the paper declares an experiment under which a claim can lose.
- **Excluded claim** marks a tempting statement that this cut does not make.

At this cut, the current evidence boundary is:

Surface	Status in this paper	What may be claimed
ABIogenesis 4.6.0-rc.3	Implemented predecessor evidence	A source-independent TypeScript GTL/ABG product line, installer, public runtime path, event/replay substrate, and bounded downstream proofs exist.
ABIogenesis 5.0 mutable source	Specified and under active realization	The complete stable-first product contract, seven-term compute target, public operator contract, Consensus free construction, self-conformance, and exact release gate are current authority.
ABIogenesis 5.0 final	Open exposure	No final-release result is claimed until one immutable cut passes the declared qualification and identity gates.
ABIogenesis 5.0.1 dogfood	Open exposure	No self-use result is claimed until installed 5.0 plus an independently released GLC product authors and qualifies the successor source through public contracts.

1.2 The internal names are not the contribution by themselves

GTL, ABG, STDO, ODD, and GLC are project names. The externally useful claims are the boundaries they make precise:

1. constructive workflow meaning belongs in a typed, published language;
2. admitted runtime truth belongs to one replayable event authority;
3. probabilistic proposal, evaluation, verification, and admission are distinct roles;
4. specification, design, code, evidence, and release identity form one traceable product claim; and
5. a product building its successor is a different claim from a runtime packaging frozen source.

The paper must stand if every internal name is replaced by its role.

2 The Problem: Patch Success Is Not Product Accountability

2.1 Software agents optimize the act of changing code

Modern software-engineering agents can browse repositories, search program structure, invoke shells, modify files, run tests, and react to failure. Systems such as SWE-agent emphasize the agent-computer interface [2]; AutoCodeRover combines language models with program-structure search and fault localization [3]; OpenHands provides a general platform for agents that interact with code, shells, browsers, and sandboxed environments [4]. Agentless demonstrates that a

simple localization-repair-validation pipeline can compete with more elaborate autonomous loops [5]. These systems establish that interface design, context selection, program representation, and executable feedback materially affect task success.

SWE-bench made this progress measurable by framing real repository issues as patch-generation tasks evaluated against tests [1]. That benchmark is valuable, but its unit of success is intentionally bounded: an issue is resolved when a patch satisfies the benchmark’s executable criteria. A production product must make additional claims. It must explain which requirement authorized the change, which design constrained it, which artifact was admitted, which actor produced it, which checks ran, which uncertainty remained, which release contains it, and how a later correction affects prior evidence.

2.2 Agent orchestration often becomes accidental product law

An agent loop necessarily contains policy. It decides what context to load, which tools are available, how failures are classified, when retries occur, which result is accepted, and when work stops. If those decisions live only in controller code, prompt templates, or model reasoning, the harness has become a private workflow language and a private authority system without naming either.

The resulting ambiguity appears at several boundaries:

- a test pass may be treated as proof of a requirement the test never encoded;
- a model judgment may be presented as deterministic verification;
- a generated artifact may be written before its schema or provenance is admitted;
- controller memory may become the only record of continuation and retry;
- a dashboard may overwrite rather than project runtime truth;
- a source-tree build may be called a released product without exact artifact identity; and
- an orchestration loop may be called self-hosting although all meaningful authoring happened outside the loop.

These are not model-capability failures. They are allocation-of-authority failures.

2.3 The research question

The governing question is:

How can probabilistic software construction be made graph-typed, replayable, evidence-bearing, source-independent, and capable of lawful correction without making the language model, harness, or downstream product a hidden authority?

The question decomposes into six research questions.

ID	Research question
RQ1	Can constructive workflow meaning be declared once as typed graph functions and consumed without controller-local reconstruction?
RQ2	Can runtime advancement and closure be derived from admitted events rather than hidden loop state?
RQ3	Can deterministic, probabilistic, and human contributions remain compositionally useful without collapsing their authority?
RQ4	Can requirement and intent pressure survive refinement, recursion, retry, and projection without disappearing at local closure boundaries?
RQ5	Can the same public product contract support native operation, downstream catalog products, and host adapters without copied orchestration?
RQ6	Can a released product and governance catalog author the product’s successor while preserving substrate/subject separation and replayable evidence?

3 Contribution

3.1 A language/runtime split for probabilistic construction

The first contribution is a strict boundary between authored constructive meaning and admitted execution truth.

GTL owns the declarations from which a lawful program is formed: graphs, nodes, graph vectors, contexts, operators, evaluators, rules, graph functions, refinement boundaries, candidate families, jobs, roles, and modules. A **GraphFunction** is the sole named callable workflow carrier. It has a typed outer contract and may realize internal graph-vector transitions, but it is not the runtime attempt, mutable workspace, or artifact produced by execution.

ABG owns admission, interpretation, graph calls, frames, attempts, workers, events, projections, continuation, retry, correction, closure, provenance, and replay. It may interpret GTL declarations; GTL does not depend on ABG runtime types. Downstream products own domain assets, vocabulary, policy, and proof interpretation. They may contribute catalog declarations and plugins, but they do not emit ABG events or own a second traversal loop.

3.2 An event-sourced epistemic runtime

The second contribution is to treat the runtime not merely as an executor but as the system’s epistemic boundary. A provider, worker, model, tool, or person can propose an effect. The effect becomes runtime truth only after ABG admits it and emits an event. Current state, assurance, continuation, gaps, and operator views are replay-derived projections over that event stream.

This yields the central runtime law:

```
provider output != runtime truth
```

```
admitted output -> event -> replay projection -> lawful consequence
```

Correction appends new truth that shadows an earlier interpretation; it does not erase history. A read model may be replaced without replacing the event authority from which it is derived.

3.3 Regime separation and admission authority

The third contribution is an explicit compute composition over three evaluator regimes:

- **F_D**: deterministic checking and mechanically total predicates;
- **F_P**: probabilistic construction and bounded semantic judgment; and
- **F_H**: attributed human selection, approval, rejection, or escalation.

The system does not infer authority from implementation style. A deterministic program can be wrong; a probabilistic evaluator can produce useful evidence; a human approval cannot turn a failed hash or schema into a pass. Selection authority is recorded separately from verification.

3.4 Obligation-conserving traversal and lawful re-entry

The fourth contribution is to make intent and requirement pressure part of the runtime contract. A local transition cannot erase a carried obligation merely because one artifact exists or one check passes. Each traversal accounts for the obligation delta, and unresolved pressure remains available to retry, refine, re-enter, reprice, block, or escalate.

This connects runtime events back to constitutional change. A defect in code may re-enter realization. A missing realization structure may re-enter design. A changed product claim may re-enter product definition or intent. The runtime does not edit those constitutional surfaces automatically; it projects typed pressure and lawful next actions for admission through the governing method.

3.5 Recursive product identity without bootstrap conflation

The fifth contribution is a recursive product taxonomy that distinguishes:

- the mutable source project being authored;
- the immutable release cut accepted from that source;
- the released product;
- an installed instance of that product; and
- an installed product acting as development substrate for another source project.

This distinction makes self-use testable. An installed predecessor packaging already-authored source is not the same as an installed product participating in authoring its successor.

3.6 The architecture at one cut

```
constitutional specification
-> ratified design
-> GTL declarations and published graph functions
-> ABG admission and interpretation
-> admitted event history
-> replay-derived state, evidence, continuation, and gaps
-> product projections and lawful re-entry proposals

F_D checks -----\
F_P constructors ----> typed proposals -> ABG admission
F_H decisions -----/

domain catalogs -> GTL meaning and policy
domain adapters <- ABG projections and public calls
```

Figure 1. Authority flows from constitutional specification into declared constructive structure. Providers contribute typed proposals at explicit regime boundaries. ABG alone admits runtime facts, and downstream views remain projections over the admitted history. A gap can propose re-entry into an upstream authority surface; it cannot rewrite that surface by itself.

4 Related Work

4.1 Repository-level issue resolution

SWE-bench [1] established repository-level issue resolution as a practical evaluation target. SWE-agent [2], AutoCodeRover [3], OpenHands [4], and Agentless [5] explore different balances among agent autonomy, interface design, program structure, tool access, and fixed workflow. Their central question is how to produce a correct repository modification efficiently.

ABIogenesis consumes those lessons but asks a different question. It does not propose a better search policy for one issue. It defines the declaration, admission, evidence, correction, and release substrate within which many search policies and worker models can operate without becoming product authority.

4.2 Role-based and multi-agent software construction

MetaGPT encodes standardized operating procedures as prompt sequences and assigns specialized roles in an assembly-line collaboration [6]. ChatDev uses communicative agents across design, coding, and testing phases [7]. More recent work on centralized asynchronous isolated delegation uses dependency-aware planning, isolated workspaces, and executable integration to improve long-horizon multi-agent work [10].

GTL can represent role-based, sequential, fan-out, fan-in, gated, and recursive work, but roles and communication are not sufficient authority. A role is a semantic capability class, a worker is an actor, a run is an execution, and an admitted event is a fact. The distinction exists so that adding more agents does not add more truth surfaces.

4.3 Harness engineering and intent-centric software engineering

AI Harness Engineering argues that software capability belongs to a model-harness-environment system and identifies task specification, context, tools, memory, task state, observability, failure attribution, verification, permissions, entropy auditing, and intervention recording as harness responsibilities [8]. Intent-centric software-engineering work similarly argues that generative AI increases the importance of specification, context, architecture, verification, provenance, governance, and human judgment [9].

These positions are close to the motivation of this paper. The additional move here is to divide the harness into a typed language and a canonical event runtime, then deny the harness authority to invent product meaning. Task state, verification, provenance, intervention, and closure are not independent controller facilities; they are admitted carriers and projections over one event record. Specification is not only task text. It is a constitutional surface with explicit re-entry law.

4.4 Requirements, traceability, and provenance

Requirement-driven agent work has begun to treat issue text as a structure to be refined rather than an instruction to execute blindly. REAgent constructs and iteratively refines issue-oriented requirements before patch generation [11]. Citation-oriented spec-driven research has separately tested whether explicit requirement identifiers make unsupported implementation claims mechanically detectable [12]. These results support the proposition that specification quality and trace structure change downstream assurance.

ABIogenesis extends the relation beyond requirement-to-code annotation. A requirement may carry stable identity, source provenance, typed relations, traversal-span coverage, evidence policy, fold state, residual pressure, and a replay-visible query identity. This brings the architecture into contact with older systems work. Build Systems à la Carte separates dependency structure from build execution and change detection [13]. Nix binds deployment to immutable content-addressed artifacts [14]. W3C PROV formalizes relations among entities, activities, and agents [15], while proof-carrying code demonstrates the stronger pattern of distributing an artifact with mechanically checkable evidence [16].

The difference is the probabilistic constructor. Much of its internal authoring process is not reproducible in the same sense as a compiler action. The architecture therefore combines exact identity and deterministic proof where available with admitted provenance and bounded semantic judgment where construction remains probabilistic.

4.5 Conceptual lineage

Two companion papers supplied vocabulary but do not serve as implementation evidence. *Emergent Reasoning* develops the view of language-model work as constraint-conditioned structured computation [17]. *Constraint-Emergence Ontology* develops the expansion/contraction framing and the separation among proposer, evaluator, verifier, and admitter [18].

This paper translates those abstractions into software-language and runtime contracts. Its claims stand or fall on ABIogenesis artifacts and the evaluation programme, not on acceptance of either companion framework. A conceptual correspondence cannot promote a runtime mechanism by analogy.

4.6 The claimed gap

This paper does not claim that existing agents lack logs, schemas, sandboxes, tests, workflows, or human gates. It claims that these facilities are usually presented as properties of an agent implementation or harness rather than as a separate graph language plus canonical runtime-truth contract spanning intent, requirements, execution, correction, and release identity.

The comparison is therefore architectural:

Work family	Primary unit	Main success surface	Authority emphasized
SWE-bench and issue-resolution agents	repository issue and patch trajectory	tests or benchmark resolution	agent/interface/workflow effectiveness
MetaGPT and ChatDev	role-based collaborative process	produced software and process outputs	role/SOP or communication structure
AI Harness Engineering	auditable agent episode	episode evidence package	harness responsibilities
ABIOgenesis GTL/ABG	published graph function and admitted traversal unit	replayable product claim from specification through release	language/runtime/product authority separation

Whether that additional structure improves outcomes enough to justify its cost is an open exposure, not a conclusion assumed from architecture.

5 Constitutional Specification

5.1 Specification is active product authority

The method begins above the runtime. Goals bound the current work wave. Intent states why the product exists and its directional constraints. Product definition states what the current source project is trying to release. Requirements decompose that product into constitutional obligations. Design defines realization structure. Code and tests realize and evaluate that design. Events record admitted execution. Projections expose current runtime truth. Observed gaps route work back to the earliest implicated authority surface.

The authority flow is:

Goals

- > Intent
- > Product Definition
- > Requirements
- > Design
- > GTL Program
- > ABG Events
- > Projection
- > Delta
- > Re-entry or Reprice

Downstream artifacts may summarize this flow, but they do not outrank it. A dashboard is a read model. A ticket records one admitted work item. A generated index is a projection. None becomes specification merely because it is easier to query.

5.2 Change begins at the smallest lawful re-entry point

The method distinguishes six change classes:

Change class	Earliest affected authority
goal_reprice	current work-wave focus
intent_reprice	direction or scope
product_reprice	product shape under stable intent
requirement_reprice	constitutional product truth
design_reframe	realization structure
realization_refactor	local implementation under stable upstream truth

The purpose is not ceremony. It prevents an implementation repair from silently changing product meaning and prevents a new product claim from being smuggled into a controller or test fixture.

5.3 Specification constrains the AI and the human

The language model does not receive authority merely because it can generate a coherent change. A human does not receive authority to override a deterministic failure merely because the human is accountable. Both act inside declared roles and through attributed admission. The constitution constrains the builder, verifier, operator, and product owner differently, but it constrains all of them.

6 GTL: The Constructive Algebra

6.1 Graph is the structural type

GTL is graph-first and composition-first. Its structural surface includes:

Carrier	Meaning
Graph	named topology of nodes and graph vectors
Node	typed local locus of graph meaning
GraphVector	admissible internal transition contract
Context	snapshot-bound external constraint
Operator	effectful action declaration
Evaluator	convergence and attestation declaration
Rule	passive declarative constraint
GraphFunction	reusable typed workflow and sole named callable carrier
RefinementBoundary	explicit boundary for lawful refinement
CandidateFamily	published alternatives under one outer contract
Job	durable semantic work contract
Role	semantic capability class
Module	publication boundary

The type distinctions are negative constraints as much as definitions. A node is not a workflow. A graph vector is not a public callable. A job is not a run. A role is not a worker. A module is not runtime truth.

6.2 Graph functions carry constructive meaning

A graph function publishes a typed outer interface and materializes a graph whose internal vectors realize that interface. The callable identity remains stable while lawful refinement may add internal structure. This permits a consumer to depend on the outer contract without inheriting the implementation topology.

The core algebra includes identity, edge construction, composition, substitution, recursion, fan-out, fan-in, gating, promotion, and same-object relations. Higher-order constructions are expected to be free constructions over those atoms. A Consensus workflow, for example, should be expressible as ordinary published graph functions, reviewer vectors, fan-out/fan-in, evaluation, bounded recursion, and an F_H escalation outcome. If it requires a Consensus-specific scheduler or service loop, the generic atom set is incomplete or the design is misplaced.

6.3 The compute algebra is distinct from graph topology

Structural graph composition and runtime compute composition are related but not interchangeable. The 5.0 contract declares a seven-term compute algebra:

```
C.of  
C.id  
C.compose  
C.edge  
workflow.C  
C.batch  
C.retry
```

C.of introduces a computation, **C.id** is identity, and **C.compose** provides associative sequencing. **C.edge** binds compute to one graph boundary. **workflow.C** is the named lift that invokes a child graph function inside a parent computation. **C.batch** groups tasks only under a declared relation to the graph topology. **C.retry** repeats the same contract under an explicit retryable-failure policy. Semantic recursion is a different operation: it opens a lawful graph-function frame with declared termination and foldback.

This surface has a deliberate status split. The 4.6.0-rc.3 language publishes all seven authored discriminants, raw admission, canonical serialization, and typed **semantic_not_realized** diagnostics. Complete generic runtime semantics for all seven terms are a 5.0 target, not predecessor evidence. A lawful syntax node proves that the language can name a gap; it does not prove that the runtime can execute the term.

6.4 Programs are configuration, not engine branches

Workflow shape is declared data. Different named compositions may coexist and be selected by reference. The engine does not infer graph shape from prompt text, handler names, controller branches, or source paths. Serialized input is admitted against the same law as native declarations; whole-program relations are checked by the semantic compiler.

This is an LLM-oriented design. The model is allowed to construct within a bounded language. Invalid native expressions should fail through the host type system where possible. Invalid serialized expressions should fail raw admission. Invalid cross-reference or whole-program relations should fail semantic compilation. Environmental or probabilistic failures remain runtime concerns.

6.5 Robustness and composability define an atom

The product evaluates each primitive against two properties:

1. **Robust**: malformed use fails closed, advancement is gated, and the result is replay-auditable.
2. **Composable**: the primitive participates in one algebra through declared boundaries rather than working only in a privileged arrangement.

Feature count is not the measure. A larger catalog of primitives with special engine branches increases the semantic surface and weakens the claim.

7 ABG: The Evidence-Bearing Runtime

7.1 Events are the only admitted write path

ABG interprets an admitted GTL program and workspace binding. Its mutable truth is an append-only event stream. Runtime projections are functions over that stream:

```
S_t = project(E_0, E_1, ... E_t)
```

A state transition is lawful only when it is represented by admitted events whose identities, basis, actor, and provenance can be replayed. Controller memory, process return values, generated reports, and cache entries may assist execution but are not rival state authorities.

7.2 The traversal unit

The bounded compute unit is one closeable traversal from **A** to **B** under a published graph function, a selected internal graph vector, a selected compute composition, execution and frame identities, admitted outputs, assurance, and consequence projection.

Informally:

```
TraversalUnit<A, B> =  
  published GraphFunction<A, B>  
  + selected GraphVector<A, B>  
  + selected compute composition  
  + admitted execution basis  
  + admitted output and evidence  
  + assurance fold  
  + consequence and replay disposition
```

This notation does not add another runtime object. It names the relation among existing carriers that must close together.

7.3 The admitted compute cycle

The runtime cycle is intentionally asymmetric. Plugins and workers propose; ABG admits and records.

```
open graph call  
  -> open invocation frame  
  -> run declared transform composition  
  -> admit transform result  
  -> emit transform events  
  -> plan evaluation set  
  -> run declared evaluation rules  
  -> admit each evaluation result  
  -> fold assurance  
  -> compute consequence proposal  
  -> admit consequence  
  -> derive traversal transition  
  -> replay continuation
```

Plugins do not emit runtime events, write ledgers, close traversals, or choose private continuation state. They return typed proposals at declared seams.

7.4 Deterministic, probabilistic, and human fibres

Each declared compute composition selects an arrangement over **F_D**, **F_P**, and **F_H**. The canonical transform-evaluate-consequence triple is one bootstrap shape, not the only lawful program.

Deterministic truth closes first wherever a total mechanical predicate exists. Probabilistic workers own open-ended construction and bounded semantic judgment. Human work is an external callout: the runtime records a pending interaction and later admits an attributed response. A held human decision is not simulated inside the engine.

7.5 Proposal, evaluation, verification, and admission are separate roles

The three compute regimes do not replace the assurance roles. A **proposer** produces candidate artifacts or actions. An **evaluator** compares a candidate with a criterion and may return a score, finding, or residual. A **verifier** checks one declared property and returns evidence. An **admitter** determines whether a typed result becomes operative runtime state.

One component may implement more than one role, but the contracts do not merge. A compiler can verify the properties it was built to see without admitting a release. A probabilistic reviewer can identify a real semantic defect without creating deterministic truth. A human can hold selection authority without making a failed checksum pass. Determinism, correctness, grounding, and selection authority are independent assurance properties.

ABG owns admission. This is the constitutional distinction that prevents a plugin from returning `verified: true`, sealing that assertion into its own evidence, and thereby manufacturing the fact that it was supposed to prove.

7.6 Closure is conjunctive

An artifact does not close a traversal merely because it exists. The current closure law combines carried convergence, fulfillment, admission, target certification, and deterministic recheck. The exact carrier names may evolve, but the conjunction is the architectural claim:

```
close =  
  carried obligations converged  
  AND declared fulfillment converged  
  AND output admitted  
  AND target contract certified  
  AND deterministic recheck passed
```

If automated assurance is absent, the system does not infer success. The default is held judgment, not silent closure.

7.7 Correction preserves history

Retry creates another attempt under declared policy. Correction records that an earlier admitted interpretation has been superseded for a current view. Re-entry routes work to an earlier graph or constitutional boundary. None of these operations deletes the event that explains how the system reached the prior state.

8 Obligation Conservation and Re-entry

8.1 Intent lineage is part of compute

The runtime function is modeled as intent-lineage preserving:


```
traverse<A, B>(intent_lineage, context, A)
  -> (B, obligation_delta)
```

The lineage binds the invocation to intent, requirements, context, authority, target contracts, materialization, prior evidence, and residual pressure. The delta accounts for each obligation. A local edge close cannot silently consume an obligation that remains unresolved at product level.

8.2 Obligation topology may be discovered during construction

Some obligations are known before work starts. Others depend on the artifact that construction reveals: generated modules imply new test surfaces; a dependency fan-out implies additional integration checks; a chosen mapping creates adversarial cases that were not enumerable from the initial request.

The system therefore distinguishes declared requirement identity from the runtime population of proof obligations. Intermediate admitted artifacts may extend that population. Completeness is judged against the extended set, not the startup count. This prevents a static checklist from proving itself complete by refusing to notice what the build discovered.

8.3 Gaps are typed outcomes

A missing declaration, unrealized semantic construct, failed deterministic gate, ambiguous requirement, unavailable capability, transport failure, and human escalation are different outcomes. Treating them all as exceptions or agent failure loses the re-entry information needed to repair the system.

Typed gaps support proportionate routing:

- malformed language returns to declaration authoring;
- missing runtime realization returns to the owning design or implementation;
- semantic ambiguity returns to probabilistic or human judgment;
- changed product meaning returns to constitutional reprice; and
- unavailable transport may retry without pretending the artifact was wrong.

9 Product, Install, and Development Product

9.1 Four identities must not collapse

The mutable repository is a source project. A release cut is an immutable boundary over accepted source. A product is the released immutable thing. An install is a stamped workspace instance of that product. When an installed product helps build another source project, it acts as a development product.

This recursive taxonomy resembles a compiler bootstrap but applies beyond compilers. It prevents statements such as “the product built itself” from collapsing the builder, mutable subject, and released artifact into one label.

9.2 Source independence is a release property

A source-blind consumer must be able to resolve, verify, install, bind, and use the released product through exact public contracts and manifests. A command that imports mutable source

or reconstructs missing declarations does not prove the release artifact.

Git reference, tarball, package identity, manifests, checksums, installed proof, and qualification evidence must identify the same bytes. Rebuilding an approximation is not evidence for the selected cut.

9.3 Catalog products extend meaning without extending authority

An installed workspace may bind compatible catalog products. They contribute published graph functions, types, overlays, policies, and domain interpretation. Their contributions enter one ABG-owned catalog under exact identity and compatibility law. Catalog presence grants no event, dispatch, continuation, or closure authority.

10 The Self-Hosting Course Correction

10.1 The original ambition

The original 5.0 direction used “ABG builds ABG” to mean that an installed governed construction stack would author the next ABIogenesis source through specification, probabilistic worker turns, deterministic admission, evidence, and convergence. This was operational dogfooding: the product and governance catalog acting as development product for their successor.

10.2 The packaging fixed point

During planning, self-hosting was reinterpreted as a two-stage equivalence: package frozen source with the predecessor runtime, package the same frozen source with the candidate runtime, and compare release-significant outputs. The design was coherent and potentially useful. It could establish that a declared build path was stable across runtime versions.

Let **I_4** be the installed predecessor, **I_5** the installed candidate, **B_5** the declared packaging program, and **S_5** the already-authored frozen source. The proof had the form:

```
I_4 + B_5 + S_5 -> C_1
I_5 + B_5 + S_5 -> C_2
C_1 equivalent_to C_2
```

It could not establish successor authoring. Meaningful construction had already happened before the fixed-point loop began. The proof was possible precisely because the source was frozen and nondeterministic fields were declared irrelevant. The loop certified packaging equivalence over pre-authored source.

This is retained as a historical negative result:

A packaging fixed point is a build-tool compatibility test. It is not evidence that the product authored its successor.

10.3 Campaign authoring and its different proof

A campaign containing probabilistic authoring cannot promise byte-for-byte reproduction by simply rerunning the model. Its evidence is instead the admitted history of what was proposed,

selected, checked, corrected, and released. Replay reproduces the truth about the artifact, not the generative trajectory that could have produced another artifact.

That distinction does not excuse weak evidence. A campaign must bind exact inputs, workers, model and tool identities where available, context cuts, events, artifacts, checks, authority decisions, and final product identity. It offers provenance and adjudicable lineage rather than deterministic reproduction of creative authoring.

10.4 The stable-first resolution

The final 5.0 constitutional decision separates baseline release from dogfood. ABIOgenesis 5.0 is authored directly under the specification method, accepted design gates, GTL admission, semantic compilation, implementation, and qualification. It must be source-independent and self-conformant, but it does not claim that installed ABIOgenesis authored 5.0.

After 5.0 is released, an independently matured GLC catalog product may bind to exact installed 5.0. The installed pair may then act as the development product for the 5.0.1 source project. That run is the first operational self-use proof. It cannot retroactively qualify 5.0.

10.5 The substrate/subject rule

During successor dogfood, the installed development product is immutable substrate and the successor repository is mutable subject. A substrate defect becomes evidence and a separately governed predecessor-line ticket. It is not patched under the campaign's feet. A missing substrate capability becomes a typed gap that may re-enter successor planning or predecessor maintenance according to ownership.

This rule prevents a self-building project from changing the judge while the trial is running.

11 Current Product Realization

11.1 Implemented predecessor surface

The TypeScript line has implemented the central language/runtime separation, package-first installation, graph-function invocation, event admission, replay-derived projection, probabilistic worker transport, result admission, assurance folding, retry and continuation carriers, output allocation, and installed qualification lanes. The immutable 4.6.0-rc.3 release-candidate cut is predecessor evidence for that substrate.

11.2 Immutable predecessor evidence

The retained 4.6.0-rc.3 release snapshot binds its claims to one clean source commit and package artifact:

Property	Recorded value
source commit	5213301cdbfd35952badf19c27519caa9e7e6968
source dirty state	false
strict semantic suite	1,430 passed / 0 failed
package entry count	810 files
package size	1,196,854 bytes
unpacked size	7,661,363 bytes
tarball SHA-256	9cffb372c0dfc00983a5d0e882efbc3d0c3ac937a56f313000f35a4473358113

The same manifest records successful strict TypeScript compilation, semantic lint, GTL authority guards, focused language-law tests, and package creation. Its C-algebra gate reports seven authored constructors, 23 reserved declaration keys, 17 inspected runner files, and 35 focused tests.

These are exact artifact facts, not a quality score. Test count does not prove independence, mutation strength, requirement coverage, or the complete 5.0 claim. The predecessor release note explicitly excludes complete runtime realization of `workflow.C`, `C.batch`, and `C.retry`, executable generic Consensus, and self-hosting. Those exclusions travel with the evidence.

11.3 Active 5.0 realization

The 5.0 product contract adds or completes:

- the full seven-term typed compute algebra and generic runtime realization;
- strict native, serialized, and semantic GTL admission;
- one versioned public SDK and thin native graph shell;
- a bounded Consensus graph function built as a free construction over public atoms;
- complete public schemas, vocabularies, operation identities, and conformance corpus;
- self-conformance, observer, and tuner evidence;
- native operation plus a bounded host projection; and
- exact immutable RC qualification and stable release.

At version 0.1 of this manuscript, this work remains active. The Consensus GTL body and several generic algebra/runtime relations are under design and compiler-probe closure. They are not reported as finished features.

11.4 Excluded product claims

The current product is bounded to one trusted developer desktop. It does not claim hostile-workstation tamper resistance, hosted multi-tenancy, IAM, cryptographic publisher authenticity, marketplace billing, autonomous scheduling, automatic wake, or unbounded ticket mutation. A generic review product and full homeostatic intent-refinement system are outside the 5.0 contract.

12 Evaluation Programme

12.1 Evaluation principle

Architecture diagrams cannot prove governance. The evaluation must compare observable system behavior under matched construction tasks and must include negative cases designed to trigger authority violations.

The primary comparison is not “GTL/ABG versus no AI.” It is the same model and tool environment under progressively stronger construction substrates.

Condition	Description
B0	prompt plus shell/editor access, final tests only
B1	fixed localization-construction-validation workflow with logs
B2	tool-rich agent harness with task state, retries, and episode package
G1	GTL-declared workflow with ABG event admission and replay
G2	full constitutional trace, obligation conservation, typed re-entry, and exact release qualification

The conditions should use matched model, repository, task, tool access, and budget where possible. Where a capability cannot be matched, the difference must be declared rather than normalized away.

12.2 Core experiments

E1: Language admission

Submit valid and invalid native and serialized graph programs. Invalid cases include missing references, wrong source/target relations, undeclared effects, ambiguous callable identity, malformed higher-order applications, and hidden program shape in opaque config.

Success: invalid structure becomes typed non-execution before worker dispatch; valid structure retains identity through serialization and semantic compilation.

E2: Single graph-function execution

Run a source-blind Hello World graph function through the public SDK and CLI.

Success: exact product and workspace identity, graph-function selection, execution basis, output, evidence, result, and replay are mutually consistent.

E3: Multi-stage mixed-regime construction

Run a workflow containing deterministic, probabilistic, and human-held boundaries.

Success: each regime contributes only through its declared role; human approval cannot override deterministic failure; probabilistic output cannot close before admission.

E4: Consensus as a free construction

Invoke the bounded Consensus graph function with multiple reviewer profiles, attributed findings, dissent, bounded recursion, and F_H escalation.

Success: the function uses only public atoms and ordinary catalog/runtime contracts; no feature-specific scheduler, service loop, ticket mutation, or private closure rule appears.

E5: Correction and replay

Admit an initially plausible but later disproven result, then record correction and resume.

Success: history remains intact, current projection changes lawfully, stale evidence cannot close the corrected traversal, and replay reproduces both the prior and current interpretations.

E6: Discovered obligations

Use a build whose intermediate artifact introduces additional modules, dependencies, or test classes.

Success: the obligation set extends from admitted artifacts; closure against the startup count is refused; every new obligation receives a disposition.

E7: Successor dogfood

Use exact installed 5.0 plus exact GLC to author and qualify the 5.0.1 source project.

Success: all authoring lands in the successor subject; the installed substrate remains immutable; capability gaps and substrate defects route to their lawful owners; final evidence binds the complete campaign to one released successor cut.

12.3 Metrics

The evaluation records:

- functional task success;
- requirement-to-design-to-code-to-test coverage;
- false-close and ungrounded-admission rate;
- completeness of actor, context, artifact, evidence, and authority lineage;
- replay agreement with live projections;
- number and class of human interventions;
- retry count and failure-class precision;
- time and compute cost;
- change-localization accuracy after a failed gate;
- portability across workers, models, and host adapters; and
- exact release-identity coherence.

Governance overhead is a first-class cost. If GTL/ABG produces equivalent assurance to a simpler harness at materially greater cost, the architecture has not earned its added structure.

12.4 Promotion law

No result promotes a stronger claim by proximity. A successful Hello World does not prove mixed-regime composition. A successful packed install does not prove source-independent successor authoring. A successful 5.0 release does not prove 5.0.1 dogfood. A successful single-project dogfood does not prove generality across domains.

13 Threats to Validity and Limitations

13.1 Single-project development

ABIogenesis is both the subject and source of the architecture. This provides deep longitudinal evidence but creates confirmation and overfitting risk. Independent downstream products and external replications are required before general claims can close.

13.2 Test volume is not claim coverage

The predecessor’s 1,430-test result is useful because it is bound to one exact artifact. The number does not establish that tests are independent, that requirements are completely covered, or that shared blind spots are absent. The evaluation must report mutation strength, negative-path reachability, requirement coverage, and installed-path evidence separately rather than using test volume as a proxy for the thesis.

13.3 Vocabulary and structural overhead

The architecture introduces a substantial type and contract vocabulary. That cost may improve precision or merely relocate complexity. The evaluation must measure authoring burden, compiler diagnostics, repair effort, and onboarding, not only runtime correctness.

13.4 Event truth is not world truth

An event stream proves what the runtime admitted, not that the admitted claim corresponds to reality. Incorrect tests, shared blind spots, or misplaced human authority can produce a coherent but wrong ledger. Grounding and adversarial evaluation remain necessary.

13.5 Trusted-desktop boundary

The current product trusts local code, filesystem, and repository transport. Digests establish coherence of selected products, not authenticity against a malicious publisher or compromised workstation.

13.6 Incomplete current result

ABIogenesis 5.0 is not released at this manuscript cut, and 5.0.1 dogfood has not run. The most ambitious claims remain open exposures. The manuscript must be revised against the immutable release evidence rather than preserving this draft’s expectations.

13.7 No claim about model cognition

The architecture governs externally observable construction and admitted evidence. It makes no claim that a language model understands the specification, maintains persistent intent, or internally executes the graph as represented.

14 Discussion

14.1 The harness becomes a language implementation

Every nontrivial agent harness contains a workflow language, even when that language is dispersed across controller branches and prompts. Naming the language makes composition, conformance, and portability testable. The cost is that hidden flexibility becomes explicit design work.

14.2 Governance is runtime structure

Documents alone do not govern an agent. Governance becomes operational when the runtime can refuse malformed structure, distinguish evidence classes, retain unresolved obligations, attribute authority, and prevent a convenient projection from rewriting history.

14.3 Reliability belongs to the composed system

The base model is neither the sole source of capability nor the sole source of risk. Reliability emerges from the composition of specification, context, language, model, tools, deterministic checks, human authority, event admission, and release discipline. Improving one component does not remove the need to state the contracts among them.

14.4 Recursive dogfooding is an architectural test

Using the product to author its successor is valuable because it stresses the same boundaries ordinary downstream use can avoid: substrate versus subject, source versus product, inherited versus newly authored authority, and provenance versus reproducibility. The test is meaningful only if those boundaries are declared before the run.

15 Conclusion

Software agents have made code modification an increasingly tractable automation problem. Governed software construction is a larger problem. It requires a stable account of what was authorized, what was constructed, what was observed, what was admitted, why work advanced, what remains unresolved, and which immutable product embodies the result.

ABIogenesis proposes one answer: a typed graph language for constructive meaning, an event-sourced runtime for admitted truth, constitutional specification with lawful re-entry, and recursive product identity that keeps the builder separate from the product being built. Probabilistic models remain powerful constructors inside this architecture. They do not become the event store, verifier, regulator, and release authority by convenience.

The architecture has already produced one important negative result. A runtime packaging frozen source is not self-hosted authoring. The corrected programme releases a stable 5.0 baseline first and tests recursive self-use on the 5.0.1 successor. That result remains to be earned.

The paper’s strongest claim is therefore conditional and testable:

AI-authored software becomes governable when constructive structure, evidence, runtime truth, and admission authority are explicit, typed, and replayable across the entire path from intent to release.

16 Appendix A. Claim Ledger

Claim	Status at v0.1	Promotion condition
GTL and ABG separate declaration meaning from runtime truth	Implemented and specified	immutable 5.0 public-contract and installed proof
Runtime state is replay-derived from admitted events	Implemented and specified	correction/replay and source-blind qualification over final cut
Regime separation prevents authority collapse	Design hypothesis with implemented guards	matched mixed-regime negative experiments
Obligation conservation reduces false closure	Design hypothesis	discovered-obligation comparison against simpler harnesses
Graph functions improve workflow portability	Open exposure	same declared program across multiple workers/hosts without copied orchestration
Consensus is a free construction over generic atoms	Specified, active design/realization	admitted body, compiler census, runtime and installed scenarios
5.0 is a complete source-independent stable product	Open exposure	final RC qualification and exact release tap
Installed 5.0 plus GLC can author 5.0.1	Open exposure	completed successor campaign and released artifact
GTL/ABG outperforms current agent harnesses on governance outcomes	Open exposure	controlled baseline comparison

17 Appendix B. Current Authority and Evidence Map

Paper concern	Repository authority
product intent	../specification/INTENT.md
product definition and language/runtime boundary	../specification/PRODUCT.md
current 5.0 delivery goal	../specification/GOALS.md
GTL requirements	../specification/requirements/gtl/
ABG requirements	../specification/requirements/abg/
product requirements	../specification/requirements/product/
common design decomposition	../build_tenants/common/design/
TypeScript realization design	../build_tenants/abiogenesis/typescript/design/
TypeScript code and tests	../build_tenants/abiogenesis/typescript/code/, ../build_tenants/abiogenesis/typescript/test_env/
5.0 stable-first reprice	../.ai-workspace/tickets/completed/T-249- constitutional-reprice-of-the-5-0-target.md
current feature trace	../.ai-workspace/tickets/completed/T-244-author- gtl5-subject-specification-seed.md
active Consensus design probe	../.ai-workspace/tickets/active/T-252-design-and- probe-consensus-gtl-free-construction.md

18 References

1. Carlos E. Jimenez et al. “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” ICLR 2024. arXiv:2310.06770.
2. John Yang et al. “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering.” 2024. arXiv:2405.15793.
3. Yuntong Zhang et al. “AutoCodeRover: Autonomous Program Improvement.” ISSTA 2024. arXiv:2404.05427.
4. Xingyao Wang et al. “OpenHands: An Open Platform for AI Software Developers as Generalist Agents.” ICLR 2025. arXiv:2407.16741.
5. Chunqiu Steven Xia et al. “Agentless: Demystifying LLM-based Software Engineering Agents.” 2024. arXiv:2407.01489.
6. Sirui Hong et al. “MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework.” 2023. arXiv:2308.00352.
7. Chen Qian et al. “ChatDev: Communicative Agents for Software Development.” ACL 2024. arXiv:2307.07924.
8. Hailin Zhong and Shengxin Zhu. “AI Harness Engineering: A Runtime Substrate for Foundation-Model Software Agents.” 2026. arXiv:2605.13357.
9. Elyson De La Cruz. “From Code-Centric to Intent-Centric Software Engineering: A Reflexive Thematic Analysis of Generative AI, Agentic Systems, and Engineering Accountability.” 2026. arXiv:2605.11027.

10. Jiayi Geng and Graham Neubig. “Effective Strategies for Asynchronous Software Engineering Agents.” 2026. arXiv:2603.21489.
11. Shiqi Kuang et al. “REAgent: Requirement-Driven LLM Agents for Software Issue Resolution.” 2026. arXiv:2604.06861.
12. Subham Panda. “Citation Discipline in Spec-Driven Development: A Cross-Model Empirical Study of Output Determinism and Automated Hallucination Detection in LLM-Generated Code.” 2026. arXiv:2606.30689.
13. Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. “Build Systems à la Carte.” *Proceedings of the ACM on Programming Languages* 2 (ICFP), 2018. doi:10.1145/3236774.
14. Eelco Dolstra, Merijn de Jonge, and Eelco Visser. “Nix: A Safe and Policy-Free System for Software Deployment.” LISA, 2004. USENIX paper.
15. Luc Moreau and Paolo Missier, eds. “PROV-DM: The PROV Data Model.” W3C Recommendation, 2013. W3C PROV-DM.
16. George C. Necula. “Proof-Carrying Code.” POPL, 1997. doi:10.1145/263699.263712.
17. Dimitar Popov. “Emergent Reasoning in Large Language Models: Soft Unification, Constraint Mechanisms, and Computational Traversal.” Zenodo, 2026. doi:10.5281/zenodo.16592399.
18. Dimitar Popov. “Constraint-Emergence Ontology: Reality as Self-Organising Constraint Network.” Zenodo, 2026. doi:10.5281/zenodo.18573722.